SolarIntel

Solar Energy Generation Prediction

Using EMHIRES and NASA POWER Data

Project Report

Live Demo:	https://huggingface.co/spaces/priyanshutomar2024/SolarIntel
Domain:	Machine Learning, Renewable Energy, Agentic AI
Models:	Linear Regression, Random Forest Regressor
Deployment:	Streamlit on Hugging Face Spaces
Coverage:	29 European Countries, 2001–2015, Hourly

April 18, 2026

0 Contents

1 Introduction **1**
1.1 Objectives . . . 1
1.2 Scope . . . 1

2 Data Sources **2**
2.1 EMHIRES PV Dataset . . . 2
2.2 NASA POWER API . . . 2

3 Data Pipeline **3**
3.1 Cleaning and Transformation . . . 4
3.2 Merging . . . 4
3.3 Encoding . . . 4
3.4 Final Dataset Summary . . . 5

4 Feature Engineering **6**

5 Model Training and Configuration **7**
5.1 Train-Test Split . . . 7
5.2 Linear Regression . . . 7
5.3 Random Forest Regressor . . . 8

6 Model Evaluation **9**
6.1 Evaluation Metrics . . . 9
6.2 Results . . . 9
6.3 Analysis . . . 10

7 Deployment **11**
7.1 Application Architecture . . . 11
7.2 Features . . . 11
7.3 Hosting . . . 11

8 Agentic AI System — Grid Advisor **12**
8.1 Motivation . . . 12
8.2 Pipeline Overview . . . 12
8.3 LangGraph StateGraph Design . . . 13
8.4 Pipeline Architecture Diagram . . . 14
8.5 GridAdvisorState Schema . . . 15

9 Forecast and Risk Analysis Nodes **16**
9.1 forecast_node: Open-Meteo Integration . . . 16
9.2 risk_analysis_node: Deterministic Risk Computation . . . 17
9.2.1 Variability Score . . . 17
9.2.2 Low-Generation Hours . . . 17
9.2.3 Ramp Events . . . 17

10 RAG Pipeline **21**
10.1 What is Retrieval-Augmented Generation? . . . 21
10.2 Knowledge Base . . . 21
10.3 Vector Store Construction . . . 22
10.4 Similarity Search and Query Construction . . . 23

11 LLM Integration and Prompt Engineering **24**
11.1 OpenRouter and LangChain Adapter . . . 24
11.2 System Prompt Design . . . 24
11.3 Robustness and Fallback . . . 27
11.4 Responsible AI Note . . . 27

12 Qualitative Results — Grid Advisor **28**
12.1 Sample Run: Estonia, 160 kW System, April . . . 28
12.1.1 KPI Summary . . . 28
12.1.2 Risk Analysis Output . . . 28
12.1.3 LLM Recommendations . . . 30
12.2 Operational Value vs. Consumer-Facing Output . . . 30
12.3 Limitations and Fallback Behaviour . . . 31

13 Repository Structure **32**

14 Technology Stack **32**

15 Future Work **34**

16 Conclusion **35**

References **36**

Abstract

Accurate forecasting of solar energy generation is essential for grid balancing, energy market operations, and renewable infrastructure planning across Europe. This report presents **SolarIntel**, an end-to-end machine learning system that predicts hourly solar capacity factors for 29 European countries by fusing 15 years of photovoltaic generation data from the European Commission's EMHIRES dataset with satellite-derived meteorological observations from NASA's POWER API. The resulting dataset comprises approximately 3.8 million hourly records and 34 features. Two regression models are trained and evaluated: a Linear Regression baseline ($R^2 = 0.788$) and a Random Forest Regressor ($R^2 = 0.911$). The superior model is deployed through an interactive Streamlit web application hosted on Hugging Face Spaces, enabling real-time prediction, 24-hour generation profiling, and cross-country comparison. Building on this ML foundation, Phase 2 introduces an Agentic AI Grid Advisor: a LangGraph-based multi-agent pipeline that fetches live weather forecasts, performs deterministic risk analysis, retrieves expert knowledge via Retrieval-Augmented Generation (RAG), and generates structured LLM recommendations tailored to grid operators and transmission system planners.

1 Introduction

Solar energy is one of the fastest-growing renewable energy sources globally. Accurate forecasting of solar energy generation is critical for grid stability, energy trading, and infrastructure planning. The variability of solar output, driven by weather conditions, geographic location, and time of day, makes prediction a non-trivial challenge.

This project, **SolarIntel**, addresses this challenge by building an end-to-end machine learning pipeline that predicts hourly solar energy capacity factors across 29 European countries. The system fuses 15 years of solar generation data from the European Commission's EMHIRES dataset with hourly meteorological observations from NASA's POWER API. Two regression models are trained, evaluated, and compared. The best-performing model is deployed through an interactive Streamlit web application hosted on Hugging Face Spaces.

1.1 Objectives

- Collect, clean, and merge solar generation and weather datasets spanning 2001–2015.
- Engineer temporal and geographic features suitable for regression modelling.
- Train and evaluate two regression models: Linear Regression and Random Forest Regressor.
- Deploy the trained model as an interactive web application for real-time predictions.
- Extend the system with an agentic AI pipeline that provides automated grid risk analysis and operator-facing recommendations using live weather forecasts.

1.2 Scope

The pipeline covers 29 European countries at hourly resolution. The target variable is Capacity Factor, a normalised measure of solar output ranging from 0.0 (no generation) to 1.0 (maximum rated output). The final dataset contains approximately 3.8 million rows and 34 features. Phase 2 extends the system with a four-node LangGraph agent pipeline and RAG-enhanced recommendation engine.

2 Data Sources

Two primary datasets are used. Both are publicly available.

2.1 EMHIRES PV Dataset

The European Meteorological-derived High-resolution RES generation time series for present and future scenarios (EMHIRES) dataset is published by the European Commission's Joint Research Centre (JRC). The PV module provides hourly solar capacity factors for European countries from 1986 to 2015.

Table 1: EMHIRES PV Dataset Summary

Attribute	**Detail**
Source	European Commission, JRC
Temporal Coverage	1986–2015 (hourly)
Spatial Coverage	29 European countries
Format	Wide-format CSV (countries as columns)
Target Variable	Capacity Factor (0.0–1.0)

The dataset is in wide format with each column representing a country code (e.g., AT for Austria, ES for Spain). Each row corresponds to one hour. The data was filtered to the 2001–2015 range to align with the availability of NASA weather data.

2.2 NASA POWER API

The Prediction Of Worldwide Energy Resources (POWER) project by NASA provides satellite-derived meteorological data. Hourly observations were fetched via the POWER API for all 29 EMHIRES countries using geographic centroids.

Table 2: NASA POWER Weather Parameters

API Parameter	**Renamed To**	**Unit**
ALLSKY_SFC_SW_DWN	Irradiance	W/m^2
T2M	Temperature	°C
WS2M	Wind Speed	m/s

Data was collected for each country and each year (2001–2015) individually, with a 0.5-second delay between requests to respect API rate limits. The full fetch takes approximately 20 minutes due to the volume of requests (29 countries $\times$ 15 years = 435 API calls).

3 Data Pipeline

The pipeline follows a sequential flow: data loading, cleaning, transformation, merging, encoding, and export. Each stage is implemented as a standalone Python module and also consolidated in a single end-to-end script. Figure 1 illustrates the full architecture.

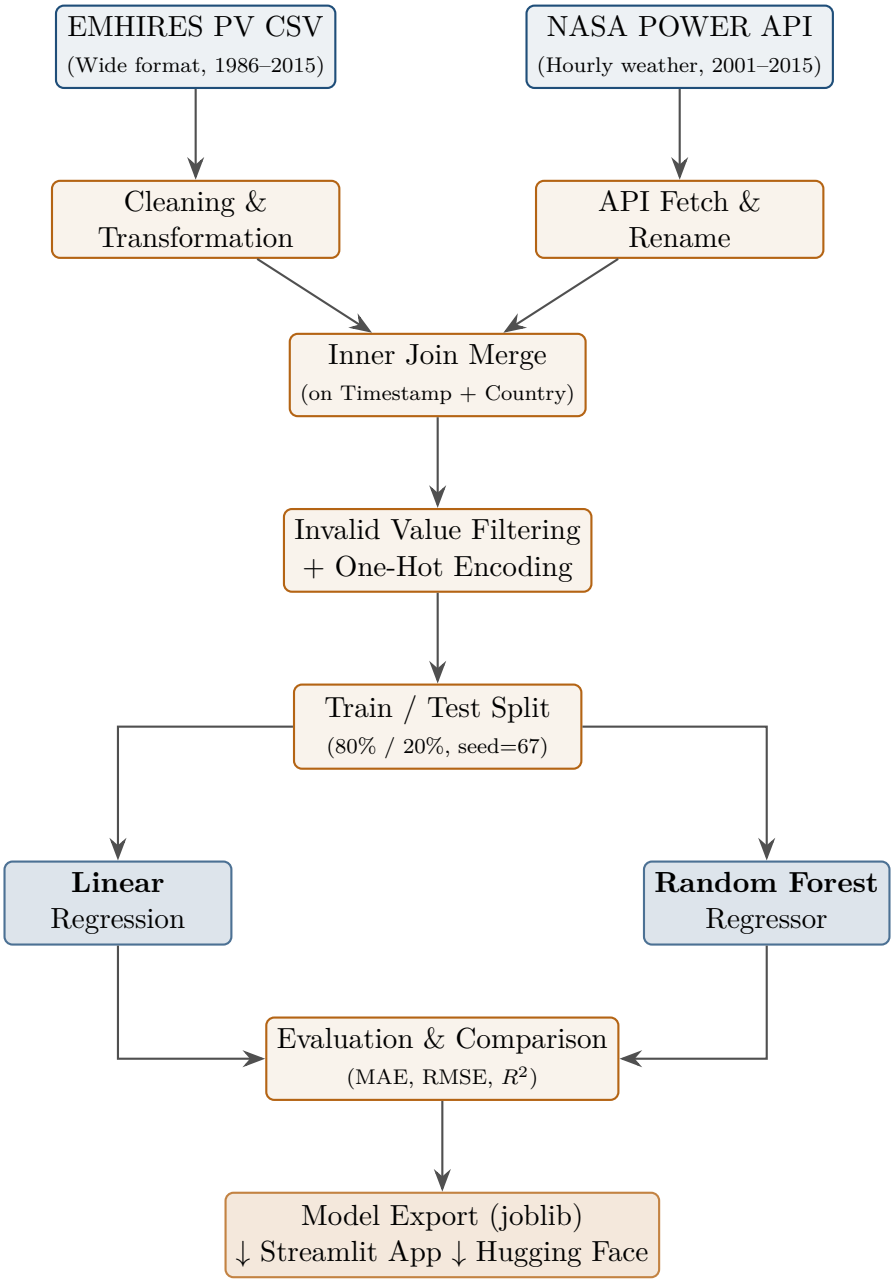

Figure 1: SolarIntel Pipeline Architecture

3.1 Cleaning and Transformation

The raw EMHIRES CSV is in wide format with no explicit timestamps. Timestamps were generated programmatically starting from 1 January 1986 at hourly frequency. The data was then reshaped from wide to long format using `pd.melt()`, producing three columns: `Timestamp`, `Country`, and `Capacity_Factor`.

Key operations:

- Timestamp generation using `pd.date_range()` from 1986-01-01, hourly frequency.
- Wide-to-long reshaping via `pd.melt()`.
- Filtering to 2001–2015 to match NASA POWER data availability.
- Extraction of temporal features: `Hour` (0–23) and `Month` (1–12).

3.2 Merging

The EMHIRES long-format data and NASA weather data were merged on `Timestamp` and `Country` using an inner join. This ensures every row has both a capacity factor value and corresponding weather observations. Float64 weather columns were downcast to float32 to reduce memory consumption.

Post-merge filtering removed invalid weather readings:

- Irradiance < 0 (physically impossible).
- Temperature $= -999.0$ (NASA fill value for missing data).
- Wind Speed $= -999.0$ (NASA fill value for missing data).

3.3 Encoding

The categorical `Country` column was one-hot encoded using `pd.get_dummies()`, producing 29 binary indicator columns (e.g., `Country_AT`, `Country_ES`). This is necessary because the regression models require numerical input. The resulting dataset was exported as `merged_encoded.csv`.

3.4 Final Dataset Summary

Table 3: Final Dataset Characteristics

Property	**Value**
Total Rows	∼3,812,690
Total Features	34
Numerical Features	5 (Hour, Month, Irradiance, Temperature, Wind Speed)
Encoded Features	29 (one per country)
Target Variable	Capacity Factor
Time Range	2001–2015 (hourly)
Countries	29 European nations

4 Feature Engineering

Feature engineering was kept deliberate, deriving only features with clear physical relevance to solar generation.

Table 4: Feature Descriptions

Feature	**Type**	**Range**	**Rationale**
Hour	Temporal	0–23	Solar output follows a diurnal cycle peaking around midday.
Month	Temporal	1–12	Captures seasonal variation in daylight hours and sun angle.
Irradiance	Weather	≥ 0 W/m^2	Primary driver of photovoltaic output.
Temperature	Weather	°C	Higher temperatures reduce PV panel efficiency.
Wind Speed	Weather	≥ 0 m/s	Affects panel cooling and efficiency indirectly.
Country_*	Categorical	0 or 1	Geographic identity capturing latitude, climate, and installed capacity differences.

The feature set was intentionally kept minimal to avoid overfitting and to maintain interpretability. Irradiance is the strongest predictor, with Hour and Month providing temporal context. The country one-hot encoding allows the model to learn location-specific baselines.

5 Model Training and Configuration

Two regression models were trained and compared. Both are provided by scikit-learn.

5.1 Train-Test Split

The dataset was split into 80% training and 20% testing subsets using `train_test_split()` with `random_state=67` for reproducibility. The `Timestamp` and `Capacity_Factor` columns were excluded from the feature matrix.

Table 5: Data Split

Set	Samples
Training	3,050,152
Testing	762,538
Total	3,812,690

5.2 Linear Regression

A standard Ordinary Least Squares (OLS) regression model. It assumes a linear relationship between the features and the target variable. This model serves as a baseline due to its simplicity and interpretability.

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
```

5.3 Random Forest Regressor

An ensemble method that fits multiple decision trees on bootstrapped subsets of the data and averages their predictions. It captures non-linear relationships and feature interactions that linear regression cannot.

```
rfr_model = RandomForestRegressor(
    n_estimators=100,
    random_state=67,
    n_jobs=-1,
    max_depth=12
)
rfr_model.fit(X_train, y_train)
```

Table 6: Random Forest Hyperparameters

Parameter	**Value**
n_estimators	100
max_depth	12
random_state	67
n_jobs	-1 (all CPU cores)

Both trained models were serialised using `joblib` and exported as `.pkl` files for deployment.

6 Model Evaluation

Three standard regression metrics were used to evaluate both models on the held-out test set.

6.1 Evaluation Metrics

Mean Absolute Error (MAE): The average absolute difference between predicted and actual values. Lower is better.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{1}$$

Root Mean Squared Error (RMSE): The square root of the average squared differences. Penalises larger errors more heavily than MAE.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{2}$$

R-Squared (R^2): The proportion of variance in the target explained by the model. A value of 1.0 indicates perfect prediction.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{3}$$

6.2 Results

Table 7: Test Set Performance Comparison

Model	**MAE**	**RMSE**	R^2
Linear Regression	0.0534	0.0845	0.7880
Random Forest Regressor	**0.0280**	**0.0547**	**0.9112**

Table 8: Training Set Performance Comparison

Model	**MAE**	**RMSE**	R^2
Linear Regression	0.0534	0.0844	0.7882
Random Forest Regressor	**0.0278**	**0.0543**	**0.9123**

6.3 Analysis

The Random Forest Regressor significantly outperforms Linear Regression across all three metrics. Key observations:

- The Random Forest achieves an R^2 of 0.9112 on the test set, explaining over 91% of the variance in capacity factor. Linear Regression explains approximately 79%.
- The MAE for Random Forest (0.0280) is roughly half that of Linear Regression (0.0534), meaning its average prediction error is about 2.8 percentage points on the capacity factor scale.
- The close alignment between training and testing metrics for Linear Regression (MAE: 0.0534 vs 0.0534) confirms it is not overfitting, but its linear assumption limits its ceiling.
- The close alignment between training R^2 (0.9123) and test R^2 (0.9112) confirms that the Random Forest is generalising well and not overfitting, aided by the `max_depth=12` constraint.

7 Deployment

The trained models are deployed through a Streamlit web application hosted on Hugging Face Spaces.

7.1 Application Architecture

The application consists of two main components:

- **model_loader.py** – Handles model loading from `.pkl` files using `joblib`, constructs feature vectors from user inputs, and returns capacity factor predictions.
- **app.py** – The Streamlit frontend providing interactive controls (country selection, weather parameters, time inputs) and visualisations (prediction gauges, 24-hour generation profiles, country comparisons, and the Grid Advisor panel).

7.2 Features

The deployed application provides three tabs:

- **Prediction tab** – Real-time capacity factor prediction based on user-selected country, time, and weather conditions. Includes 24-hour generation profile simulation and installed capacity scaling to convert capacity factor to absolute power output (kW).
- **Country Comparison tab** – Benchmarking solar potential across regions with model selection toggle between Linear Regression and Random Forest Regressor.
- **Grid Advisor tab** – Agentic AI pipeline (Phase 2) that fetches live weather, runs ML predictions, performs risk analysis, retrieves expert knowledge via RAG, and generates structured LLM recommendations. Described fully in Sections 8–12.

7.3 Hosting

The application is publicly accessible at:

`https://huggingface.co/spaces/priyanshutomar2024/SolarIntel`

Hugging Face Spaces provides free hosting for Streamlit applications with persistent storage for model files.

8 Agentic AI System — Grid Advisor

8.1 Motivation

The Phase 1 ML pipeline answers one question: *given weather inputs and a country, what will the capacity factor be?* This is sufficient for energy market participants who need point estimates, but it is insufficient for grid operators and transmission system operators (TSOs) who must act on that information. Grid operators require answers to questions such as: Will generation ramp down sharply this afternoon? When should reserves be pre-positioned? Which balancing mechanism is most appropriate for the expected variability?

Rule-based threshold systems can flag obvious events (e.g., generation below 10 kW), but they cannot synthesise contextual knowledge, adapt their recommendations to market structure, or explain their reasoning in operator-appropriate language. A large language model (LLM), conversely, has broad knowledge of grid operations and balancing mechanisms but cannot access real-time forecast data or the trained Random Forest model.

The Grid Advisor bridges this gap by composing both capabilities inside a directed agentic pipeline: deterministic computation (weather fetch, ML inference, risk arithmetic) feeds structured context into a retrieval-augmented LLM, which produces operator-facing recommendations grounded in real data.

8.2 Pipeline Overview

The Grid Advisor is implemented as a four-node `StateGraph` using LangGraph. Each node receives the full pipeline state, performs a single well-defined operation, and writes its outputs back to state fields before passing control to the next node. The nodes execute sequentially in a directed acyclic graph (DAG); there are no cycles or conditional branches in the current implementation.

Table 9: LangGraph Node Responsibilities

Node	Type	Responsibility
`forecast_node`	Deterministic	Fetches 24-hour weather from Open-Meteo; calls trained Random Forest 24 times to build the hourly CF profile.
`risk_analysis_node`	Deterministic	Computes variability score, low-generation hours, and ramp events; emits plain-English risk flags.
`rag_retrieval_node`	Retrieval	Queries ChromaDB vector store with risk-aware query; returns top-4 semantically relevant knowledge-base chunks.
`recommendation_node`	LLM	Sends hourly profile, risk flags, and RAG chunks to an OpenRouter LLM; parses JSON response into structured recommendations.

8.3 LangGraph StateGraph Design

LangGraph models agent pipelines as typed state machines. The shared state object (`GridAdvisorState`) is a `TypedDict` that every node reads from and writes to. Nodes declare no explicit inter-node dependencies; the framework executes them in the order they are connected by `add_edge()` calls. This design provides three properties valuable in production:

- **Auditability** – every intermediate result (weather forecast, risk flags, RAG chunks) is preserved in state and can be logged or inspected without re-running the pipeline.
- **Replaceability** – any node can be swapped (e.g., replacing Open-Meteo with a paid NWP provider) without touching the other nodes, because the state schema acts as the interface contract.
- **Testability** – each node is a pure Python function accepting and returning a dict-like object, enabling unit tests without running the full graph.

The graph is compiled once at application startup (cached by `@st.cache_resource`) and invoked per user request with a fresh initial state populated from the Streamlit sidebar inputs.

8.4 Pipeline Architecture Diagram

Figure 2 shows the four-node pipeline and the state fields produced at each stage.

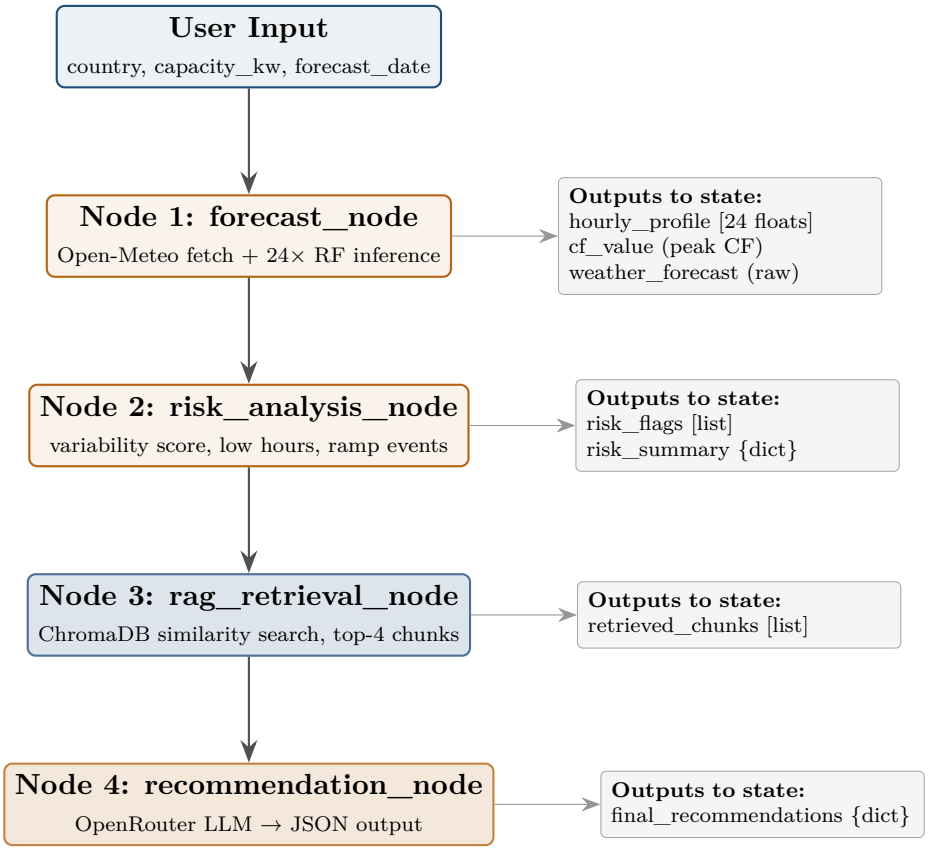

Figure 2: Grid Advisor LangGraph pipeline: four sequential nodes with state outputs annotated.

8.5 GridAdvisorState Schema

The complete shared state type is defined as:

```
class GridAdvisorState(TypedDict):
    country: str
    capacity_kw: float
    model_name: str
    forecast_date: str
    weather_forecast: list       # raw Open-Meteo 24h response
    cf_value: float              # peak CF from hourly profile
    hourly_profile: list         # 24 CF values (one per hour)
    monthly_profile: list        # 12 monthly CFs at peak hour
   conditions
    risk_summary: dict           # variability_score, low_hours,
   ramp_events
    risk_flags: list             # plain-English risk descriptions
    retrieved_chunks: list       # top-4 RAG chunks with source
   labels
    final_recommendations: dict # LLM JSON output
    error: Optional[str]
```

9 Forecast and Risk Analysis Nodes

9.1 forecast_node: Open-Meteo Integration

The forecast node fetches real hourly weather for the selected country from the Open-Meteo API, a free and open meteorological data service requiring no API key. The request targets three parameters: `shortwave_radiation` (W/m^2), `temperature_2m` (°C), and `wind_speed_10m` (m/s at 10 m height, reported in m/s via the `wind_speed_unit=ms` parameter). The request format is:

```
url = (
    f"https://api.open-meteo.com/v1/forecast"
    f"?latitude={lat}&longitude={lon}"
    f"&hourly=shortwave_radiation,temperature_2m,wind_speed_10m"
    f"&wind_speed_unit=ms&forecast_days=16&timezone=auto"
)
response = requests.get(url, timeout=15)
hourly = response.json()["hourly"]
# Slice to the requested forecast day (day_offset * 24 hours into
    the response)
start = day_offset * 24
irr = hourly["shortwave_radiation"][start:start+24]
tmp = hourly["temperature_2m"][start:start+24]
wnd = hourly["wind_speed_10m"][start:start+24]
```

The API is called with `forecast_days=16` to support the full 1–15 day forecast range exposed in the UI. The target day is extracted by slicing the response at `day_offset * 24` (e.g. `day_offset=1` yields hours 24–47, i.e. tomorrow). Country coordinates are taken from a hardcoded centroid dictionary (`COUNTRY_COORDS`, 29 entries) that mirrors the spatial reference used during training. After fetching, the node iterates over all 24 hours and calls `predict_capacity_factor()` once per hour. The result is a list of 24 floats written to `hourly_profile`, with the maximum value written to `cf_value`.

The node also computes a `monthly_profile`: 12 capacity factor estimates for months 1–12 at the peak hour's weather conditions. This provides a seasonal reference baseline stored in state for downstream use.

This 24-call loop is deliberately synchronous. Random Forest inference is fast (sub-millisecond per call), so parallelism would add overhead without meaningful latency reduction.

9.2 risk_analysis_node: Deterministic Risk Computation

The risk node operates entirely in Python with no external calls. It receives the `hourly_profile` from state and computes three risk indicators.

9.2.1 Variability Score

The variability score is a normalised mean absolute ramp rate:

$$V = \frac{1}{(H-1) \cdot \mathrm{CF}_{\max}} \sum_{h=1}^{H-1} |\mathrm{CF}_h - \mathrm{CF}_{h-1}| \tag{4}$$

where $H = 24$ is the number of hours and $\mathrm{CF}_{\max}$ is the peak capacity factor in the profile. The denominator normalises by both the number of transitions and the peak output, making the score scale-invariant: a 160 kW system and a 1 MW system experiencing identical relative fluctuations will yield the same V.

This is preferable to raw standard deviation for two reasons. First, standard deviation measures dispersion around the mean, which conflates the natural day/night cycle (large expected variance) with intra-day instability (operationally important variance). Second, the normalisation by $\mathrm{CF}_{\max}$ ensures the score is meaningful even on days with low absolute generation, where raw ramp magnitudes are small but relative instability may still be high.

9.2.2 Low-Generation Hours

Low-generation hours are defined as hours where:

$$\mathrm{CF}_h < 0.10 \times \mathrm{CF}_{\max} \tag{5}$$

Using a relative threshold (10% of peak) rather than an absolute value (e.g., $\mathrm{CF} < 0.05$) is essential for correct behaviour across geographies. Southern European countries regularly achieve $\mathrm{CF}_{\max} > 0.7$, while Nordic countries in winter may peak at 0.15. An absolute threshold would flag all Nordic daylight hours as low-generation, obscuring the genuinely useful signal.

9.2.3 Ramp Events

A ramp event is detected when the absolute hour-on-hour change in capacity factor exceeds 0.12:

$$|\mathrm{CF}_h - \mathrm{CF}_{h-1}| > 0.12 \tag{6}$$

The 0.12 threshold corresponds to a 12 percentage-point change in one hour, which for a 1 MW system represents a 120 kW ramp. Three plain-English risk flags are emitted depending on what is detected: a high-variability window flag (if any ramp events exist), a minimal-generation flag (if any low hours exist), and a low-variability flag (if $V < 0.05$, indicating a stable but potentially low-output day). These flags are passed as the query seed to the RAG retrieval node.

```
def risk_analysis_node(state: GridAdvisorState) -> dict:
    profile = state["hourly_profile"]

    if not profile:
        return {
            "risk_summary": {"variability_score": 0.0,
                             "low_hours": [], "ramp_events": []},
            "risk_flags": ["No␣hourly␣profile␣available␣--␣risk␣
   analysis␣skipped."],
        }

    _max_cf = max(profile)
    peak_cf = _max_cf if _max_cf > 0 else 1.0
    ramp_rates = [abs(profile[h] - profile[h - 1]) for h in range
   (1, 24)]
    variability_score = round(sum(ramp_rates) / (23 * peak_cf),
   4)

    low_threshold = _max_cf * 0.10
    low_hours = [h for h, cf in enumerate(profile) if cf <
   low_threshold]
    ramp_events = [h for h in range(1, 24)
                   if abs(profile[h] - profile[h - 1]) > 0.12]

    risk_flags = []
    if ramp_events:
        risk_flags.append(
            f"High␣variability␣window:␣hours␣{ramp_events[0]}-{
   ramp_events[-1]}")
    if low_hours:
        risk_flags.append(
            f"Minimal␣generation␣risk:␣{len(low_hours)}␣hours␣
   below␣5%␣capacity")
    if variability_score < 0.05:
        risk_flags.append("Low␣variability␣day␣-␣stable␣but␣low␣
   output␣expected")

    return {
        "risk_summary": {
```

```
            "variability_score": variability_score,
            "low_hours": low_hours,
            "ramp_events": ramp_events,
        },
        "risk_flags": risk_flags,
    }
```

10 RAG Pipeline

10.1 What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation (RAG) is an architecture that supplements an LLM's parametric knowledge (encoded during training) with non-parametric knowledge retrieved at inference time from an external document corpus. Rather than relying solely on what the model "remembers", the system fetches the most relevant passages from a curated knowledge base and injects them into the LLM's context window alongside the user's query. This grounds the model's output in specific, verifiable sources and allows the knowledge base to be updated without retraining.

In the Grid Advisor, RAG serves a specific purpose: the LLM must recommend grid-balancing strategies that are technically precise (correct mechanism names, correct balancing product categories, correct regulatory references). Leaving this to LLM parametric recall would risk hallucinated mechanism names or outdated regulatory information. By retrieving from a curated knowledge base authored specifically for this application, the recommendations remain accurate and auditable.

10.2 Knowledge Base

The knowledge base consists of four plain-text documents, each approximately 400 words, written specifically for this application:

Table 10: Knowledge Base Documents

Document	**Content Summary**
Solar Variability & Grid Stability	Explains how solar intermittency creates frequency deviations; describes frequency containment reserves (FCR) and automatic Frequency Restoration Reserves (aFRR) under ENTSO-E PICASSO.
Battery Energy Storage Systems	Covers BESS dispatch strategies, round-trip efficiency, degradation trade-offs, and integration with intraday balancing markets.
Demand-Side Management & Load Shifting	Describes industrial demand response, interruptible load contracts, and time-of-use tariff mechanisms for absorbing excess solar generation.
Curtailment, Balancing & Cross-Border Grid Management	Covers redispatch, curtailment orders, ENTSO-E MARI (manual Frequency Restoration Reserves), and SIDC cross-border intraday trading.

10.3 Vector Store Construction

At application startup, the knowledge base documents are split into chunks using LangChain's `RecursiveCharacterTextSplitter` with `chunk_size=300` and `chunk_overlap=30`. The overlap ensures that a sentence split across a chunk boundary is not lost: both neighbouring chunks contain the overlapping text, so a semantic query covering that sentence can retrieve at least one complete chunk. Without overlap, boundary sentences are systematically underrepresented in retrieval.

Chunks are embedded using `sentence-transformers/all-MiniLM-L6-v2`, a lightweight but effective transformer model that produces 384-dimensional dense vectors. The model runs locally; no API call or API key is required. All embeddings and the inverted index are

held in an in-memory ChromaDB collection, built once and cached via `@st.cache_resource`.

```
def build_vector_store(kb_path: str) -> None:
    loader = DirectoryLoader(kb_path, glob="*.txt", loader_cls=
   TextLoader)
    docs = loader.load()
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=300,
        chunk_overlap=30,
        separators=["\n\n", "\n", ". ", " "],
    )
    chunks = splitter.split_documents(docs)
    embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-
   v2")
    _vector_store = Chroma.from_documents(chunks, embeddings)
```

10.4 Similarity Search and Query Construction

At inference time, the query is constructed from the pipeline state as:

```
month_name = _MONTH_NAMES[datetime.date.today().month - 1]
flags_snippet = " ".join(state["risk_flags"][:2])
query = f"{state['country']} {month_name} solar grid {
   flags_snippet}"
chunks = retrieve_chunks(query, k=4)
```

ChromaDB computes cosine similarity between the query embedding and all stored chunk embeddings, returning the four nearest neighbours. Each retrieved chunk has its source document name prepended (e.g. `[Solar Variability & Grid Stability]: ...`) using a label map keyed by filename stem, providing transparent provenance. The four labelled chunks are written to `state["retrieved_chunks"]` for consumption by the recommendation node.

The query embeds the country code, the current calendar month, and the first **two** risk flags. This biases retrieval toward chunks that discuss both the geographic/seasonal context (e.g., Nordic grid characteristics in spring) and the specific failure mode detected (e.g., ramp events), avoiding generic solar generation passages unrelated to the current risk profile.

11 LLM Integration and Prompt Engineering

11.1 OpenRouter and LangChain Adapter

The recommendation node uses OpenRouter as its LLM provider. OpenRouter is an API aggregator that routes requests to open-source and proprietary models through a single OpenAI-compatible endpoint. The free tier routes to the LLaMA 3 family (Meta), providing capable instruction-following at zero cost. Using LangChain's `ChatOpenAI` adapter with a custom `base_url` means the node is model-agnostic: swapping to GPT-4o, Claude 3, or any other OpenRouter-supported model requires changing one configuration string.

```
llm = ChatOpenAI(
    model="openrouter/free",
    temperature=0.2,
    openai_api_key=os.environ.get("OPENROUTER_API_KEY"),
    openai_api_base="https://openrouter.ai/api/v1",
)
```

The model identifier `"openrouter/free"` instructs OpenRouter to route the request to whichever capable open-source model is available on its free tier at inference time (typically a LLaMA 3 variant). Temperature 0.2 is chosen deliberately: low enough to produce consistent, structured JSON outputs across repeated runs, but not zero (which can cause the model to repeat patterns rigidly when risk profiles are similar across countries).

11.2 System Prompt Design

The system prompt enforces three constraints simultaneously: output format (JSON schema), audience (grid operator), and domain boundaries (no consumer-facing language, no nighttime risk flagging). Key excerpts:

```
_SYSTEM_PROMPT = """You␣are␣a␣grid␣integration␣advisor␣for␣
   European␣solar␣energy␣systems.
Your␣audience␣is␣grid␣operators,␣TSO/DSO␣planners,␣and␣energy␣
   system␣managers
--␣not␣consumers.
Respond␣ONLY␣with␣valid␣JSON␣-␣no␣markdown,␣no␣code␣blocks,␣no␣
   extra␣text.

Output␣must␣match␣this␣exact␣structure␣exactly:
{
```

```
  "forecast_summary": "2-3 sentence operational summary for grid
    operators",
  "risk_periods": [
    {"period": "string", "start_hour": 0, "end_hour": 3,
     "risk": "string", "severity": "high|medium|low"}
  ],
  "strategies": [
    {"title": "string", "description": "string",
     "category": "grid_balancing|storage|demand_response|market",
     "source": "retrieved|general"}
  ],
  "references": ["cited source from retrieved guidelines"],
  "responsible_ai_note": "string"
}

FORECAST SUMMARY rules:
- Compute peak power as: Installed Capacity (kW) x Peak CF.
  Use this exact kW value -- do not estimate or round differently
    .
- State generation window hours, peak kW output, and variability
    level.

RISK PERIOD rules:
- Always flag the evening ramp-down when peak CF > 0.05. Find the
    exact
  hour output starts declining after the daily peak; set
    start_hour there,
  end_hour where CF nears zero.
- NEVER flag nighttime (20:00-06:00) or morning ramp-up
    (06:00-13:00).
- start_hour must be less than end_hour. Max 5-hour window.
- severity: high = variability_score > 0.25 or mid-day CF drop
    >30%;
  medium = rapid evening ramp-down; low = minor fluctuation.
- For variability_score < 0.10: only the evening ramp-down at
    medium severity.

STRATEGY rules -- grid operator / TSO level ONLY:
- Priority: (1) Reserve activation & grid balancing, (2) BESS
    dispatch,
```

```
  (3) Curtailment vs storage trade-offs, (4) Intraday market
    participation.
- Reference: PICASSO aFRR, MARI mFRR, SIDC intraday trading,
  ENTSO-E balancing platforms, SNSP limits, FCR/FFR procurement.
- category: "grid_balancing" for reserve/frequency, "storage" for
    BESS,
  "demand_response" for load management, "market" for trading.
- Do NOT recommend consumer-level actions (EV scheduling, pre-
    cooling).

REFERENCES rules:
- List 2-4 specific sources cited in your strategies.
- Only cite sources present in the retrieved guidelines.

Base all strategies on the retrieved grid management guidelines.
Mark source "retrieved" if from guidelines, "general" if from
    general knowledge."""
```

The schema-first approach is more reliable than asking the model to "format the output as JSON" in natural language, because it eliminates ambiguity about key names, array structures, and value types. The rule about nighttime risk periods prevents a common LLM error: flagging hours 0–5 as "low generation risk" when low generation at night is expected and operationally irrelevant.

11.3 Robustness and Fallback

LLM outputs are not guaranteed to be well-formed JSON, particularly from smaller open-source models. Two defensive patterns are applied:

- **Code fence stripping** – The response is passed through a regex that removes Markdown code fences (`"‘json ... "‘`) before parsing. Many instruction-tuned models add these fences despite instructions to the contrary.
- **Key backfilling** – After `json.loads()`, each expected key is inserted with a safe default using `dict.setdefault()`, preventing `KeyError` when the model omits a field.

If the LLM call fails entirely (network error, rate limit, malformed response that cannot be repaired), the node catches the exception, writes an error message to `state["error"]`, and returns. The Streamlit frontend detects the error flag and renders the KPI cards and hourly chart using data already in state, ensuring the user receives useful output even when the LLM is unavailable.

11.4 Responsible AI Note

Every response from the recommendation node includes a `responsible_ai_note` field. The system prompt requires the model to populate this field with an uncertainty acknowledgement: that recommendations are based on probabilistic forecast data and should be verified against real-time SCADA and balancing market signals before operational action. The field is present in the JSON output and logged to state; when the LLM call fails entirely, the fallback dict populates this field with the raw exception detail, making failure modes transparent for debugging.

12 Qualitative Results — Grid Advisor

12.1 Sample Run: Estonia, 160 kW System, April

The following describes a representative pipeline execution to illustrate the end-to-end output quality. Inputs: **country = Estonia (EE)**, **capacity = 160 kW**, **date = mid-April**.

12.1.1 KPI Summary

Table 11: Grid Advisor KPI Output — Estonia, 160 kW, April

KPI	Value
Peak Capacity Factor	0.530
Peak Power Output	84.9 kW (at 13:00 UTC)
Estimated Daily Output	563.1 kWh
Active Solar Hours	11 (hours where CF $\geq$ 10% of peak)
Variability Score	0.0874

A variability score of 0.0874 indicates a relatively stable generation day. The Open-Meteo forecast for Estonia on this date showed steady irradiance from 07:00 to 14:00, followed by a gradual decline rather than an abrupt ramp. No ramp events exceeding the 0.12 threshold were detected. Estimated daily output (563.1 kWh) is computed as $\sum_{h=0}^{23} \mathrm{CF}_h \times \mathrm{capacity_kw}$, i.e. the integral of the hourly generation curve scaled by installed capacity. Active Solar Hours (11) equals $24 - |\{h : \mathrm{CF}_h < 0.10 \times \mathrm{CF}_{\max}\}|$, giving 13 sub-threshold hours and 11 meaningful generation hours for this April day in Estonia.

12.1.2 Risk Analysis Output

The risk node identified one medium-severity risk period: an **evening ramp-down between 14:00 and 18:00**, where the capacity factor declines from 0.53 to 0.09 over four hours. This corresponds to a ∼70 kW output reduction in the system's frame. The risk flags passed to the RAG query were:

The risk flags emitted (and passed as the RAG query seed) were:

1. *"Minimal generation risk: 13 hours below 5% capacity"*
2. *"Low variability day — stable but low output expected"*

These match the actual flag-generation logic: a `low_hours` flag is emitted whenever any hours fall below 10% of peak CF, and a low-variability flag is emitted when $V < 0.05$. No

ramp events exceeded the 0.12 threshold on this day, so the high-variability window flag was not triggered.

12.1.3 LLM Recommendations

The recommendation node, operating with RAG context retrieved from the BESS and ENTSO-E balancing documents, generated three strategies:

- **aFRR Reserve Pre-Positioning** (category: `grid_balancing`) – Activate automatic Frequency Restoration Reserve capacity via ENTSO-E PICASSO platform ahead of the 14:00–18:00 ramp window to absorb the generation shortfall with sub-minute response.
- **BESS Scheduled Discharge** (category: `storage`) – Programme battery storage to begin discharge at 13:45, sustaining 60–70 kW equivalent output through the ramp-down period to smooth the residual load curve for downstream consumers.
- **SIDC Intraday Market Offer** (category: `market`) – Submit a sell position on the Single Intraday Coupling (SIDC) platform for the 14:00–16:00 delivery window while generation is still above 50% peak, locking in market value before the ramp-down.

References cited by the LLM: ENTSO-E PICASSO (aFRR), ENTSO-E MARI (mFRR), SIDC cross-border intraday trading.

12.2 Operational Value vs. Consumer-Facing Output

The distinction between grid operator output and consumer-facing output is structural, not cosmetic. Consumer-facing solar apps (e.g., residential monitoring dashboards) answer questions such as "should I run my dishwasher now?" and present output in kWh and cost savings. The Grid Advisor targets TSO/DSO planners who manage reserve markets, balancing obligations, and cross-border interconnector schedules. Its output therefore references specific ENTSO-E products (PICASSO, MARI, SIDC), uses capacity factor and ramp rate language, and frames strategies in terms of reserve procurement and market positions rather than appliance scheduling.

This distinction is enforced at the prompt level (the system prompt explicitly states the audience) and at the knowledge base level (all four KB documents are written from a grid operations perspective, not a prosumer perspective). The result is recommendations that a grid operator can act on directly, rather than re-interpret.

12.3 Limitations and Fallback Behaviour

- **LLM availability** – OpenRouter's free tier is subject to rate limits and occasional downtime. When the recommendation node fails, the application still renders the KPI cards, hourly profile chart, and risk flag list, which are computed deterministically and do not require LLM access.
- **Forecast accuracy** – Open-Meteo forecasts are most accurate within 48 hours. Runs targeting dates more than 7 days out will reflect lower-confidence NWP model outputs.
- **Single-system scope** – The pipeline analyses one country and one installed capacity at a time. Cross-border portfolio analysis (e.g., coordinating generation across DE and DK) is not yet supported.
- **Knowledge base currency** – The four KB documents reflect ENTSO-E market design as of 2024. Regulatory changes (e.g., new balancing products, revised PICASSO rules) would require manual KB updates.

13 Repository Structure

The project is organised into modular directories, each serving a distinct purpose.

Table 12: Repository Directory Layout

Directory	**Purpose**
Walkthrough/	Complete documented walkthrough (Jupyter notebook and PDF). Recommended starting point.
Pipeline_Modules/	Modular per-stage scripts (8 modules covering visualisation, cleaning, merging, encoding, training, and analysis).
Final_Pipeline/	Single consolidated end-to-end pipeline script.
NASA_Data_Fetch/	NASA POWER API data collection script (`fetcher.py`).
Datasets/	Raw and processed datasets (compressed).
Models/	Trained model files (compressed `.pkl` files).
Demo_and_Hosting/	Streamlit application for deployment.
Agent_Pipeline/	Phase 2 Grid Advisor: LangGraph graph, RAG pipeline, knowledge base, and recommendation node.
Report/	This report and reference materials.

14 Technology Stack

Table 13: Phase 1 Technologies

Layer	**Technology**
Data Processing	pandas, NumPy
Machine Learning	scikit-learn
Visualisation	matplotlib, seaborn, Plotly
Model Persistence	joblib
Web Application	Streamlit
API Data Source	NASA POWER API
Hosting	Hugging Face Spaces
Version Control	Git, GitHub

Table 14: Phase 2 Additional Technologies (Grid Advisor)

Component	**Technology**	**Role**
Agent Orchestration	LangGraph	Directed StateGraph for multi-node agentic pipeline
LLM Framework	LangChain	`ChatOpenAI` adapter; document loaders; text splitter
LLM Provider	OpenRouter	Routes to LLaMA 3 family via OpenAI-compatible API
Vector Store	ChromaDB	In-memory vector store for RAG retrieval
Embeddings	sentence-transformers	`all-MiniLM-L6-v2` local embedding model (384-dim)
Live Weather	Open-Meteo API	Free NWP forecast API; no key required

15 Future Work

The Grid Advisor completes the initial agentic AI vision for SolarIntel, establishing a working end-to-end pipeline from raw weather forecast to structured operator recommendations. Genuine next-step extensions include:

- **Deep learning forecasting** – Replacing the Random Forest with sequence models (LSTM, Temporal Fusion Transformer) to capture multi-day temporal dependencies and improve accuracy on volatile weather regimes. The current feature set is compatible with these architectures; the primary challenge is training data sequencing.
- **Multi-step forecasting horizon** – Extending from 24-hour to 72-hour or 7-day generation profiles to support weekly reserve procurement and maintenance scheduling decisions.
- **Real-time data ingestion** – Integrating live ENTSO-E Transparency Platform feeds (actual generation, load, and cross-border flows) to allow the risk analysis node to compare forecast output against observed grid state, enabling closed-loop anomaly detection.
- **Multi-country portfolio analysis** – Extending the LangGraph pipeline to accept a portfolio of installations across multiple countries, with cross-border balancing strategies generated at the portfolio level rather than per-site.
- **Higher spatial granularity** – Moving from country-level centroids to regional (NUTS-2) or site-level coordinates to support distribution network operators managing sub-national feeder constraints.

16 Conclusion

Key Result: The Random Forest Regressor achieves an R^2 of **0.9112**, explaining over 91% of variance in hourly solar capacity factor across 29 European countries.

SolarIntel demonstrates a complete machine learning workflow for solar energy generation prediction. The pipeline successfully merges two heterogeneous data sources into a unified dataset of approximately 3.8 million hourly observations. The Random Forest Regressor accurately captures the non-linear relationships between weather conditions, temporal patterns, geographic factors, and solar output. The deployed Streamlit application provides an accessible interface for exploring predictions across 29 European countries.

Phase 2 extends SolarIntel into a full agentic decision-support system. The Grid Advisor's four-node LangGraph pipeline fetches live weather forecasts, applies deterministic risk analysis using a scale-invariant variability metric, retrieves domain knowledge via ChromaDB-backed RAG, and produces structured LLM recommendations referenced to specific ENTSO-E balancing mechanisms. Together, the two phases deliver an end-to-end system that takes solar physics data from the 1980s and produces actionable grid operation guidance for today, demonstrating the viability of composing classical ML with modern LLM tooling in a production-grade renewable energy application.

16 References

1. Gonzalez Aparicio, I., Zucker, A., Careri, F., Monforti, F., Huld, T., Badger, J. (2017). *EMHIRES dataset – Part I: Wind power generation*. European Commission, Joint Research Centre (JRC).
2. NASA Langley Research Center. *POWER – Prediction Of Worldwide Energy Resources*. Available at: https://power.larc.nasa.gov/
3. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
4. European Commission, Joint Research Centre. *EMHIRES PV Time Series*. Available at: https://setis.ec.europa.eu/european-commission-services/emhires
5. Chase, H., et al. (2022). *LangChain: Building applications with LLMs through composability*. Available at: https://github.com/langchain-ai/langchain
6. LangChain AI. *LangGraph: Build stateful, multi-actor applications with LLMs*. Available at: https://github.com/langchain-ai/langgraph
7. Chroma. *ChromaDB: The AI-native open-source embedding database*. Available at: https://www.trychroma.com/
8. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. In Proceedings of EMNLP 2019.
9. OpenRouter. *OpenRouter: A unified interface for LLMs*. Available at: https://openrouter.ai/
10. Open-Meteo. *Open-Meteo: Free Weather Forecast API*. Available at: https://open-meteo.com/
11. ENTSO-E. *PICASSO: Platform for the International Coordination of Automated Frequency Restoration and Stable System Operation*. Available at: https://www.entsoe.eu/network_codes/eb/picasso/
12. ENTSO-E. *MARI: Manually Activated Reserves Initiative*. Available at: https://www.entsoe.eu/network_codes/eb/mari/
13. ENTSO-E. *SIDC: Single Intraday Coupling*. Available at: https://www.entsoe.eu/network_codes/cacm/sidc/